# Diseño del software: interfaces, arquitectura, patrones y datos

*Proyecto formativo · GPS Guardián Escolar*

Equipo del proyecto GPS Guardián Escolar

5 de noviembre de 2026

**Versión documental:** 2.5.7 · **Estado:** versión editable preparada para subida y revisión de QA/arquitectura.

## Tabla de contenido

Resumen

Arquitectura del sistema

Diseño de interfaces y navegación

Patrones y principios aplicados

API y contrato entre capas

Modelo de datos lógico y físico

Seguridad y límites de confianza

Diseño de despliegue y estados del ciclo de vida

Criterios de conformidad y pendientes de diseño

Referencias y trazabilidad (APA 7.ª edición)

**Corte y alcance de evidencia.** Revisión documental 2.5.7, 5 de noviembre de 2026. La descripción del **release de referencia** utiliza los commits de main observados en Backend (1022058d), Frontend (3d9f59fa), DB (9278b1dc) e Infra (131e9895). Se inspeccionaron además las ramas dev/develop más recientes y los suplementos documentales 2.5.6, pero **sus cambios no se consideran integrados a producción por esa sola razón**. La revisión es estática: no se ejecutaron en esta sesión pruebas, despliegues ni validaciones en teléfono. Los resultados operativos requieren evidencia real.

## Resumen

El diseño se fundamenta en la estructura del cliente Expo, la API Spring Boot modular y las migraciones PostgreSQL/Liquibase identificadas en el corte de main. Describe vistas de componentes, interfaces por rol, contratos, persistencia, decisiones y limitaciones. Se corrigen descripciones anteriores que trataban PATCH /api/trips/{id}/status como endpoint vigente, ya que no figura en TripController.java revisado.

## 1. Arquitectura del sistema

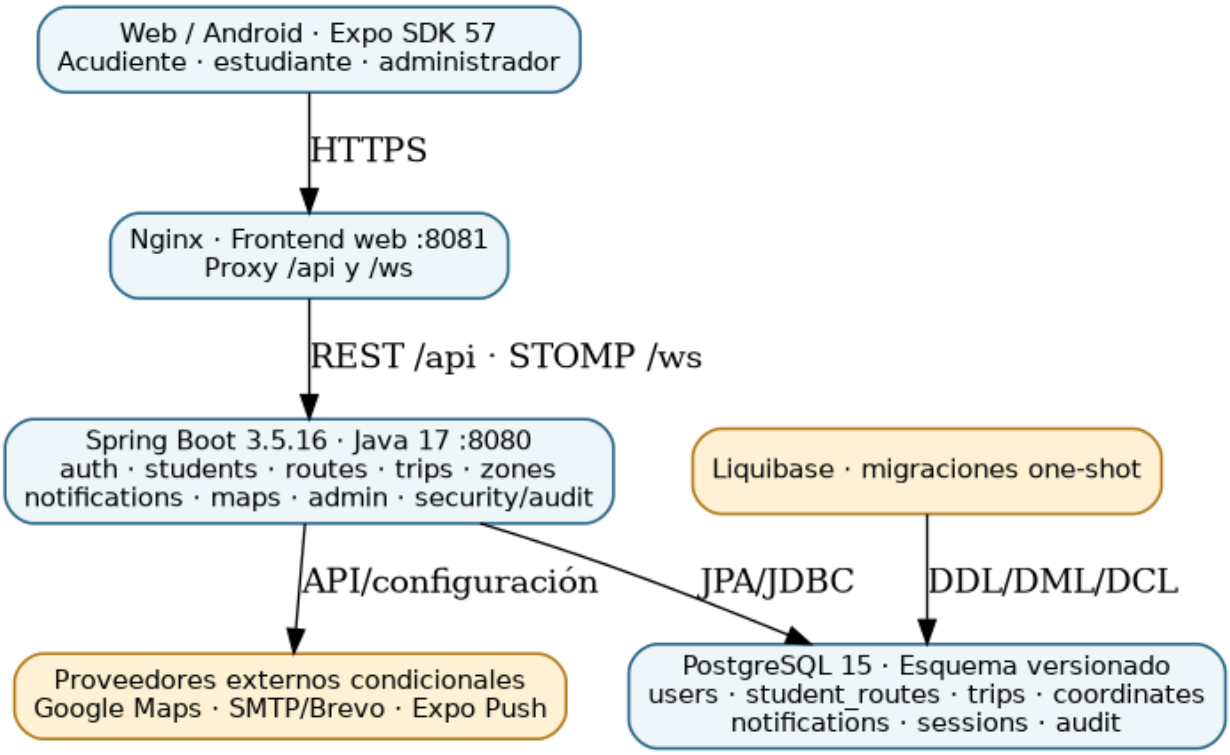



Figura 1. Componentes y dependencias de implementación

La solución usa **arquitectura cliente-servidor en capas** con backend organizado por dominio. La presentación se desarrolla con Expo Router; los controladores REST ejecutan validación y autorización; servicios de

| | | |
|---|---|---|
| | | negocio coordinan entidades; los repositorios JPA acceden al esquema PostgreSQL. WebSocket/STOMP complementa la API con la emisión de coordenadas; los proveedores Maps/Correo/Expo Push son dependencias opcionales o condicionadas por configuración. |
| Capa | Responsabilidad | Artefacto verificable |
| Presentación | Autenticación, estudiantes, rutas, seguimiento, zonas, alertas | app/, features/, components/ |
| Comunicación | Cliente HTTP, STOMP, resolución de endpoints y sesión | services/api.js, services/socket.js, config/ |
| API | Entradas REST, validación DTO, control de permisos | modules/*/controller |
| Dominio | Validación de asignación, sesión, viaje, alerta, vinculación | modules/*/service |
| Persistencia | CRUD y consultas del negocio | modules/*/repository y PostgreSQL |
| Entorno | Arranque ordenado y monitorización | Docker-Infra-Guardian-Escolar/docker-compose.yaml |

**Decisión D-01:** mantener autoridad en la API para acciones sensibles, aunque el cliente oculte un botón. **Decisión D-02:** no modificar tablas con ddl-auto update; Liquibase es la fuente de cambios de esquema.

## 2. Diseño de interfaces y navegación

El frontend organiza autenticación en app/(auth)/ (welcome, login, register, verify-email, verify-code, confirm-login, forgot-password, reset-password) y zona autenticada en app/(tabs)/ (index, student, tracking, zones, history, notifications, profile y admin); agrega scan-qr.js y student-dashboard.js.

Pantalla/flujo

Usuario

Entradas

Salidas y validación

Registro/login

Cuenta nueva o existente

Datos y credenciales

Validación, correo/2FA, sesión; errores diferenciados

Estudiantes

Acudiente/admin autorizado

Datos, foto opcional

Listado paginado y edición; controles por permiso

Vínculo QR

Acudiente + dispositivo infantil

QR/código y confirmación

Vínculo autenticado; no usar códigos ajenos

Rutas

Gestor autorizado

Ruta, paradas, origen/destino

Asignaciones y ruta preferida

Tracking

Acudiente autorizado

Selección de estudiante/trayecto

Posición y última actualización; degradación visible

Zonas seguras

Usuario autorizado

Centro y radio en metros

Mapa, estado de zona, guardado validado

Notificaciones

Cuenta autorizada

Preferencias y lectura

Bandeja, leídas, canales según disponibilidad

Administración

Rol ADMIN

Consultas operativas

Dashboard, alertas, auditoría y errores

**Diseño móvil:** usar expo-camera para QR y expo-location para ubicación. La cámara, ubicación en primer/segundo plano, notificaciones y consumo de batería requieren permisos específicos y pruebas con una build nativa. Una exportación web no demuestra seguimiento nativo continuo.

**Usabilidad y accesibilidad:** mostrar fecha/hora del último dato GPS, rechazo comprensible de código, confirmación explícita antes de vincular, estado de sesión, control táctil legible y acciones restringidas por rol; no indicar que el viaje «finalizó correctamente» si la API solo confirma baja/cancelación.

## 3. Patrones y principios aplicados

| Patrón/principio | Evidencia de diseño | Beneficio |
| --- | --- | --- |
| Componentes por dominio | modules.auth/students/routes/trips/zones/maps/notifications/admin | Separación de responsabilidades |
| Controller–Service–Repository | Clases homólogas en el backend | Aislamiento de entrada, negocio y persistencia |
| DTO y validación declarativa | TripDtos, StudentDtos, anotaciones @Valid | Contratos y errores consistentes |
| Inyección de dependencias | Spring y constructores | Reemplazo de servicios y pruebas |
| Interceptor/filtro de seguridad | Spring Security y verificación STOMP CONNECT | |

| | |
|---|---|
| Autorización API y mensajes | |
| Proxy/Facade | |
| /api/maps/** frente a Google Maps | |
| Ocultar clave servidor y consolidar errores | |
| Caché y resiliencia | |
| Caffeine, Resilience4j, rate-limit Maps | |
| Evitar abuso y limitar caídas externas | |
| Context/Provider | |
| Estado compartido de sesión del cliente | |
| Reducir acoplamiento entre pantallas | |
| Publicación/suscripción | |
| STOMP /topic/trips/{id}/coordinates | |
| Actualizar mapa sin sondeo permanente | |
| Soft-delete | |
| deleted_at de trayectos y entidades aplicables | |
| Trazabilidad sin borrar físicamente por defecto | |

El término «patrón adoptado» se basa en clases/dependencias observadas, no en un certificado de rendimiento o madurez arquitectónica.

## 4. API y contrato entre capas

Convenciones: JSON sobre HTTPS, prefijo /api, bearer JWT para la mayoría de operaciones, UUID para identificadores, paginación page/size con base cero para estudiantes, viajes y coordenadas. Respuestas de error gestionadas por la API; registrar el código exacto al probar cada escenario.

Recurso

Ejemplos verificados en controladores

Advertencia

Auth

POST /api/auth/register, /login, /logout, GET /api/auth/me, /verify-email

Confirmación y 2FA según contexto

Estudiantes

GET/POST /api/students, PUT/DELETE /api/students/{id}

@PreAuthorize por permiso/ADMIN

Rutas preferidas

PUT /api/students/{id}/preferred-route/{routeId}

Requiere ruta asignada activa

Vínculo

POST /api/students/{id}/link-code, POST /api/students/link-device

Validación backend, no confiar en la UI

Viajes

GET /api/trips, POST /api/trips, GET /api/trips/active?studentId=...

201, paginación, 204 sin activo

Coordenadas

POST/GET /api/trips/{tripId}/coordinates

Solo registrar con viaje activo

Borrado de viaje

DELETE /api/trips/{tripId}

Baja lógica; activo → CANCELED

Mensajes en vivo

WS /ws, /topic/trips/{id}/coordinates

CONNECT autenticado

Salud

GET /api/health

Health no sustituye pruebas E2E

**Inconsistencia retirada:** PATCH /api/trips/{id}/status era citado en documentación histórica; **no se encuentra en TripController del release inspeccionado**. El resto de operaciones debe contrastarse con OpenAPI exportado en el entorno concreto antes de declarar contrato definitivo.

## 5. Modelo de datos lógico y físico

Dominio

Tablas/familias principales

Relaciones y controles

Identidad

users, roles, permissions, user_roles, role_permissions, user_sessions

Control de acceso y sesiones

Estudiante

students, student_guardians, student_devices, emergency_contacts

Pertenencia, vínculo, acceso compartido

Transporte

routes, stops, student_routes

Ruta asignada/activa y preferida

Viaje GPS

trips, coordinates

Coordenadas vinculadas a trayecto, estados

Ubicación contextual

safe_zones

Radio y georreferencia

Mensajes

notifications, notification_receipts, notification_settings, user_devices

Entrega y estado de lectura separados

Control

audit_logs, activity_logs, error_logs

Investigación y soporte

**Migración 014:** incorpora student_routes.is_preferred y un índice único parcial para que una asignación activa preferida sea inequívoca por estudiante. **Relaciones útiles:** estudiante ↔️ rutas mediante student_routes; estudiante ↔️ acudiente mediante student_guardians; estudiante ↔️ viaje ↔️ coordenadas; usuario ↔️ recibos de notificación. Los diagramas ER/MR del ZIP 1.0/2.0 son referencias históricas: deben actualizarse con la migración 014 si se publican como definitivos.

La base es evolucionada mediante Liquibase (01_ddl, 02_dml, 03_dcl, changelog, 05_rollbacks), evitando ediciones retroactivas de changesets aplicados. Las consultas de tablas y restricciones deben validarse en el ambiente destino.

## 6. Seguridad y límites de confianza

**Autenticación:** credenciales, verificación de correo, posible 2FA y tokens JWT. **Autorización:** permisos de controlador más validaciones de propiedad/gestión de estudiante en servicios. **Datos de menores:** acceso mínimo, códigos efímeros de vinculación, no compartir coordenadas entre cuentas, protección de secretos y política de retención. **Seguridad móvil:** SecureStore donde aplique; las variables EXPO_PUBLIC_* forman parte del binario y no pueden contener secretos de servidor.

## 7. Diseño de despliegue y estados del ciclo de vida

El docker-compose.yaml define database saludable → liquibase exitoso → backend saludable → frontend saludable. PostgreSQL usa el rol de bootstrap para migraciones; la API utiliza rol propio de aplicación de menor privilegio gps_guardian_app con APP_DB_PASSWORD. La inicialización de administrador requiere contexto explícito; el ciclo de vida de secretos no debe incluir Git.

El estado de un viaje se representa en el enum (PENDING, IN_PROGRESS, COMPLETED, CANCELED); el código de servicio revisado demuestra creación directa en IN_PROGRESS y cancelación al eliminar activamente. Documentar otras transiciones solo tras localizar endpoint y evidencias.

## 8. Criterios de conformidad y pendientes de diseño

Quedan sujetos a pruebas y cotejo: captura de pantallas de la build aprobada, exportación actual de OpenAPI, migración en BD del release, experiencia Android QR/GPS en segundo plano, consistencia de notificaciones, mockups frente a pantallas reales y validación de diagramas históricos. En la revisión no se proporcionan números medidos de disponibilidad, latencia ni consumo energético.

## Referencias y trazabilidad (APA 7.ª edición)

GPS Guardian Escolar. (2026a). *Backend-Guardian-Escolar* [Código fuente, rama main, commit 1022058d]. GitHub. https://github.com/GPS-Guardian-Escolar/Backend-Guardian-Escolar/tree/1022058d545a9e7936edbff75950034ec4f98e88

GPS Guardian Escolar. (2026b). *Fronted-GuardianEscolar-* [Código fuente, rama main, commit 3d9f59fa]. GitHub. https://github.com/GPS-Guardian-Escolar/Fronted-GuardianEscolar-/tree/3d9f59fa3e91b5ecc5b8ae86591187546b966668

GPS Guardian Escolar. (2026c). *DB-Structure-Guardian-Escolar* [Migraciones SQL, rama main, commit 9278b1dc]. GitHub. https://github.com/GPS-Guardian-Escolar/DB-Structure-Guardian-Escolar/tree/9278b1dcb224301536d6158357f74daee74b20ba

GPS Guardian Escolar. (2026d). *Docker-Infra-Guardian-Escolar* [Configuración de despliegue, rama main, commit 131e9895]. GitHub. https://github.com/GPS-Guardian-Escolar/Docker-Infra-Guardian-Escolar/tree/131e98954430e95e9108174eded9896b3f0d5272

GPS Guardian Escolar. (2026e). *Docs-Guardian-Escolar: entregables 2.5.5 y suplementos 2.5.6* [Documentación del proyecto]. GitHub. https://github.com/GPS-Guardian-Escolar/Docs-Guardian-Escolar

**Fuentes primarias relevantes:** TripController.java, TripService.java, StudentController.java, LocationAvailabilityService.java, WebSocketConfig.java, application.yml, package.json, docker-compose.yaml y 01_ddl/04_alter/014_preferred_student_route.sql, según el documento. Las referencias a rutas son comprobables en los repositorios indicados.